



Xi'an Jiaotong-Liverpool University

西交利物浦大學

Lab 1: Introduction to Signals

ADS203TC Digital Signal Processing

Dr. Pascal Lefèvre

Contents

Introduction	2
1 Part 1: Theoretical Foundations of Signals	2
1.1 The Digital Signal Processing (DSP) System	2
1.2 Signal Definition and Classification	3
2 Part 2: Exercises	4
2.1 Generating and Visualizing Basic Signals (easy)	4
2.2 Continuous vs. Discrete Signal Representation (intermediate)	5
2.3 The Effect of Sampling and Aliasing (advanced)	7
2.4 Signal Addition and Noise (advanced)	9

Introduction

Objectives

This laboratory session is designed to provide a practical introduction to the fundamental concepts of signals. By the end of this 2-hour session, you will be able to:

- Understand the classification of signals (continuous/discrete, deterministic/random).
- Utilize core Python libraries (NumPy, Matplotlib) for signal processing tasks.
- Generate, visualize, and manipulate basic discrete-time signals.
- Observe and understand the practical implications of the sampling theorem, including aliasing.
- Synthesize composite signals and model the addition of noise.

Prerequisites

- Basic proficiency in Python programming.
- A working Python environment with a Jupyter Notebook instance.
- The following packages installed: `numpy`, `scipy`, `matplotlib`. You can install them via `pip`:
`pip install numpy scipy matplotlib jupyter`

Please launch a new Jupyter Notebook to complete the exercises. It is recommended to begin by importing the necessary libraries in the first cell:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import signal
```

1 Part 1: Theoretical Foundations of Signals

This section provides a formal overview of signals and their properties, which will form the basis for the practical exercises.

1.1 The Digital Signal Processing (DSP) System

Most practical DSP systems serve as an interface between the analog world and digital computation. A typical system architecture is depicted in Figure 1.

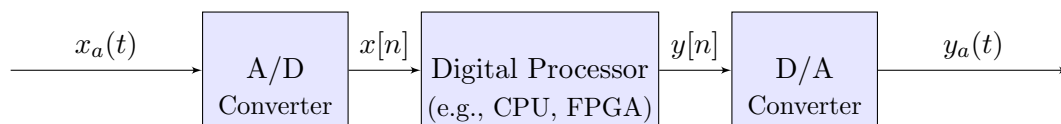


Figure 1: A typical Digital Signal Processing system pipeline.

The key components are:

- **Analog-to-Digital Converter (A/D):** Converts a continuous-time, continuous-amplitude analog signal $x_a(t)$ into a discrete-time, discrete-amplitude digital sequence $x[n]$. This involves two main processes: sampling (discretizing time) and quantization (discretizing amplitude).

- **Digital Processor:** Performs mathematical operations on the digital sequence $x[n]$ to produce a new sequence $y[n]$. This is where filtering, transformation, and analysis algorithms are implemented.
- **Digital-to-Analog Converter (D/A):** Converts the processed digital sequence $y[n]$ back into an analog signal $y_a(t)$.

1.2 Signal Definition and Classification

Definition. A signal is formally defined as a function of one or more independent variables, which conveys information about a physical phenomenon. For instance, a speech signal can be modeled as a function of time, $s(t)$, representing air pressure variations.

Dimensionality. Signals are classified based on the number of independent variables:

- **One-Dimensional (1D):** Function of a single variable, e.g., audio $s(t)$.
- **Two-Dimensional (2D):** Function of two variables, e.g., a static image $I(x, y)$.
- **Multi-Dimensional (M-D):** Function of multiple variables, e.g., video $V(x, y, t)$.

Time and Amplitude Domains. A critical classification in DSP is based on the nature of the time variable and the signal's amplitude.

Continuous-Time vs. Discrete-Time: A *continuous-time* signal $x(t)$ is defined for all values of time $t \in \mathbb{R}$. An *discrete-time* signal $x[n]$ is defined only at discrete integer instants $n \in \mathbb{Z}$. The sequence $x[n]$ is often obtained by sampling a continuous signal, $x[n] = x_a(nT_s)$, where T_s is the sampling period.

Continuous-Valued vs. Discrete-Valued: A *continuous-valued* signal can take on any real (or complex) value. A *discrete-valued* signal's amplitude is restricted to a finite set of values. This is the result of quantization.

A signal that is discrete in both time and amplitude is formally termed a **digital signal**. In theoretical analysis, we often work with discrete-time, continuous-valued signals to simplify the mathematics, treating quantization error as a separate noise source.

Deterministic vs. Random.

- **Deterministic Signal:** Its behavior over time is fully specified by a mathematical formula, e.g., $x(t) = A \cos(2\pi ft)$. There is no uncertainty about its value at any point in time.
- **Random Signal:** Its behavior cannot be predicted precisely. These signals are described using statistical properties like mean, variance, and probability distributions. Examples include thermal noise in electronic circuits or background radio emissions.

2 Part 2: Exercises

2.1 Generating and Visualizing Basic Signals (easy)

Objective. To become familiar with generating discrete-time signals using NumPy and visualizing them with Matplotlib. We will generate and plot a simple sinusoidal signal.

Mathematical Background. A continuous-time sinusoid is given by:

$$x_a(t) = A \cos(2\pi f_0 t + \phi) \quad (1)$$

where A is the amplitude, f_0 is the frequency in Hertz, and ϕ is the phase in radians. When this signal is sampled with a sampling frequency f_s (or sampling period $T_s = 1/f_s$), the resulting discrete-time signal is:

$$x[n] = x_a(nT_s) = A \cos(2\pi f_0 nT_s + \phi) = A \cos(\omega_0 n + \phi) \quad (2)$$

where n is the integer sample index and $\omega_0 = 2\pi(f_0/f_s)$ is the normalized discrete-time angular frequency in radians per sample.

Tasks.

1. Define the signal parameters:
 - Amplitude $A = 1.0$
 - Signal frequency $f_0 = 5$ Hz
 - Sampling frequency $f_s = 100$ Hz
 - Phase $\phi = \pi/4$ radians
 - Duration = 1 second
2. Generate a time vector **n** for the discrete samples. Use `np.arange` or `np.linspace`. The vector should cover the 1-second duration. Hint: The number of samples will be equal to the sampling frequency if the duration is 1 second.
3. Generate the discrete-time signal **x_n** using the formula for $x[n]$ and the parameters defined above. Use `np.cos` and `np.pi`.
4. Visualize the signal using `matplotlib.pyplot`. Create two subplots:
 - **Plot 1:** Use `plt.plot(n, x_n)` to show the signal with points connected by lines. This gives an impression of the underlying continuous signal.
 - **Plot 2:** Use `plt.stem(n, x_n)` to visualize the individual discrete samples as "lollipopops." This is the proper way to represent a discrete-time sequence.
5. Label your axes clearly (e.g., "Sample Index (n)" and "Amplitude") and provide a title for each plot.

```

1 # 1. Define parameters
2 fs = 100 # Sampling frequency in Hz
3 f0 = 5   # Signal frequency in Hz
4 A = 1.0  # Amplitude
5 phi = np.pi / 4 # Phase
6 duration = 1.0 # seconds
7
8 # 2. Generate sample indices

```

```

9 num_samples = int(duration * fs)
10 n = np.arange(num_samples)
11
12 # 3. Generate the discrete-time signal
13 x_n = A * np.cos(2 * np.pi * f0 * n / fs + phi)
14
15 # 4. Plotting
16 plt.figure(figsize=(12, 6))
17
18 # Subplot 1: Line plot
19 plt.subplot(2, 1, 1)
20 plt.plot(n, x_n)
21 plt.title('Sinusoidal Signal (Line Plot)')
22 plt.xlabel('Sample Index (n)')
23 plt.ylabel('Amplitude')
24 plt.grid(True)
25
26 # Subplot 2: Stem plot
27 plt.subplot(2, 1, 2)
28 plt.stem(n, x_n)
29 plt.title('Sinusoidal Signal (Stem Plot)')
30 plt.xlabel('Sample Index (n)')
31 plt.ylabel('Amplitude')
32 plt.grid(True)
33
34 plt.tight_layout()
35 plt.show()

```

Questions.

1. How many full cycles of the sinusoid do you observe in the 1-second interval? Does this match the specified signal frequency f_0 ?
2. Modify the sampling frequency f_s to 20 Hz. What changes do you observe in the plots? How does the shape of the signal in the line plot compare to the original?

2.2 Continuous vs. Discrete Signal Representation (intermediate)

Objective. To understand and visualize the fundamental difference between a continuous-time analog signal and its discrete-time representation obtained through sampling. This exercise highlights that a discrete signal is a sequence of values, not a continuous curve.

Mathematical Background. A continuous-time signal, such as $x_a(t)$, is defined for every instant of time $t \in \mathbb{R}$. In contrast, its discrete-time counterpart, $x[n]$, is defined only at specific, uniformly spaced time instants, $t = nT_s$, where T_s is the sampling period and $n \in \mathbb{Z}$ is the sample index.

$$x[n] = x_a(t)|_{t=nT_s} = x_a(nT_s) \quad (3)$$

In a computational environment, a true continuous signal cannot be perfectly stored or plotted. We simulate it by computing its value at a very large number of closely spaced points, creating a visually smooth curve. This exercise will contrast this high-density representation with a lower-density, truly discrete representation.

Tasks.

1. Define the parameters for a sinusoidal signal:
 - Signal frequency $f_0 = 3$ Hz
 - Amplitude $A = 1.5$
 - Duration = 2 seconds
2. Define two different sampling frequencies:
 - A very high rate to simulate the continuous signal: $f_{s,\text{high}} = 1000$ Hz.
 - A lower rate for the discrete signal: $f_{s,\text{discrete}} = 25$ Hz.
3. Generate the "continuous" signal: create a time vector `t_cont` and a signal vector `x_cont` using the high sampling rate.
4. Generate the discrete signal: create a discrete time vector `t_disc` and a signal vector `x_disc` using the lower sampling rate.
5. Create a single plot that overlays both signals.
 - Plot the "continuous" signal (`x_cont`) as a smooth line.
 - Plot the discrete signal (`x_disc`) using `plt.stem`, which is the appropriate visualization for a discrete sequence.
6. Ensure the plot includes a title, axis labels, and a legend to distinguish between the two representations.

```

1 # 1. Define signal parameters
2 f0 = 3.0          # Frequency of the sinusoid in Hz
3 A = 1.5           # Amplitude
4 duration = 2.0    # Duration in seconds
5
6 # 2. Define sampling rates
7 fs_high = 1000.0  # High sampling rate for "continuous" feel
8 fs_discrete = 25.0 # Lower sampling rate for the discrete signal
9
10 # 3. Generate the "continuous" signal
11 # Create a fine-grained time vector
12 t_cont = np.arange(0, duration, 1/fs_high)
13 x_cont = A * np.sin(2 * np.pi * f0 * t_cont)
14
15 # 4. Generate the discrete signal
16 # Create a coarser time vector for the samples
17 t_disc = np.arange(0, duration, 1/fs_discrete)
18 x_disc = A * np.sin(2 * np.pi * f0 * t_disc)
19
20 # 5. Plotting
21 plt.figure(figsize=(12, 6))
22
23 # Plot the "continuous" signal as a line
24 plt.plot(t_cont, x_cont, 'b-', label=f'Continuous-Time Signal (Simulated at {
    fs_high} Hz)')
25
26 # Plot the discrete signal using a stem plot
27 plt.stem(t_disc, x_disc, linefmt='r-', markerfmt='ro', basefmt='r-',
28         label=f'Discrete-Time Signal (Sampled at {fs_discrete} Hz)')
29
30 plt.title('Comparison of Continuous and Discrete Signal Representations')

```

```

31 plt.xlabel('Time (s)')
32 plt.ylabel('Amplitude')
33 plt.legend()
34 plt.grid(True)
35 plt.show()

```

Questions.

1. Examine the resulting plot. What information about the signal is explicitly captured by the discrete representation (the red stems)? What information is lost between the samples?
2. In the mathematical background, it is stated that $x[n] = x_a(nT_s)$. How does the plot visually confirm this relationship?
3. Modify the discrete sampling frequency $f_{s,\text{discrete}}$ to 7 Hz. The Nyquist rate for a 3 Hz signal is 6 Hz, so taking 7 Hz is adequate (we take this for granted for the sake of learning by experimenting but later, we formally introduce the sampling theorem and the Nyquist rate, or have a look at 2.3). How does the visual representation of the discrete signal change? Now, set $f_{s,\text{discrete}}$ to 5 Hz (below the Nyquist rate) and re-run the code. What do you observe in the stem plot, and how does it relate to the underlying blue curve?

2.3 The Effect of Sampling and Aliasing (advanced)

Objective. To practically demonstrate the Nyquist-Shannon sampling theorem and the phenomenon of aliasing that occurs when a signal is undersampled. *Sampling will be formally introduced in Week 3*, but this exercise serves as an early hands-on exploration of the concept.

Mathematical Background. The Nyquist-Shannon sampling theorem states that to perfectly reconstruct a continuous-time signal from its samples, the sampling frequency f_s must be strictly greater than twice the maximum frequency component of the signal ($f_{\max}$).

$$f_s > 2f_{\max} \quad (4)$$

If this condition is not met ($f_s \leq 2f_{\max}$), a phenomenon called **aliasing** occurs. High-frequency components in the original signal "impersonate" lower frequencies in the sampled version. The apparent frequency, or aliased frequency f_{alias} , can be found by:

$$f_{\text{alias}} = |f_0 - k \cdot f_s| \quad (5)$$

where k is an integer chosen such that $0 \leq f_{\text{alias}} \leq f_s/2$.

Tasks.

1. Consider a continuous-time signal $x_a(t) = \cos(2\pi \cdot 40t)$. The signal frequency is $f_0 = 40$ Hz. According to the Nyquist theorem, what is the minimum sampling rate required to avoid aliasing?
2. Let's sample this signal with a frequency $f_s = 50$ Hz, which is below the Nyquist rate. Calculate the expected aliased frequency f_{alias} .
3. **Implementation:**

- To simulate the "true" continuous signal, generate it with a very high sampling rate (e.g., $f_{s,\text{high}} = 1000$ Hz). Plot this signal over a 0.2-second duration. This will be our reference.
- Now, generate the undersampled signal by sampling at $f_s = 50$ Hz over the same duration.
- On the same axes, plot the high-rate reference signal as a line and the undersampled points as distinct markers (e.g., red 'o's). This will help you visualize how the samples fall on the original waveform.
- In a new plot, visualize only the undersampled signal by connecting its points with a line. Does the frequency of this new waveform match the aliased frequency you calculated in step 2?

```

1 # Parameters
2 f0 = 40.0 # Original frequency
3 fs_under = 50.0 # Undersampling frequency
4 duration = 0.2
5
6 # Generate high-resolution signal for reference
7 fs_high = 1000.0
8 t_high = np.arange(0, duration, 1/fs_high)
9 x_high = np.cos(2 * np.pi * f0 * t_high)
10
11 # Generate undersampled signal
12 t_under = np.arange(0, duration, 1/fs_under)
13 x_under = np.cos(2 * np.pi * f0 * t_under)
14
15 # Plotting to show aliasing
16 plt.figure(figsize=(12, 6))
17
18 # Plot 1: Original signal with undersampled points
19 plt.subplot(2, 1, 1)
20 plt.plot(t_high, x_high, label=f'Original Signal ({f0} Hz)')
21 plt.plot(t_under, x_under, 'ro', label=f'Sampled at {fs_under} Hz')
22 plt.title('Original Signal and Undersampled Points')
23 plt.xlabel('Time (s)')
24 plt.ylabel('Amplitude')
25 plt.legend()
26 plt.grid(True)
27
28 # Plot 2: Reconstructed from undersampled points
29 plt.subplot(2, 1, 2)
30 plt.plot(t_under, x_under, 'r-o')
31 # Calculate expected alias frequency: |40 - 50| = 10 Hz
32 f_alias = 10.0
33 plt.title(f'Apparent Signal from Undersampling (Alias at {f_alias} Hz)')
34 plt.xlabel('Time (s)')
35 plt.ylabel('Amplitude')
36 plt.grid(True)
37
38 plt.tight_layout()
39 plt.show()

```

Question. In the second plot, count the number of cycles over the 0.2-second interval. Calculate the frequency from your observation. Does it match your theoretical calculation for f_{alias} ?

2.4 Signal Addition and Noise (advanced)

Objective. To synthesize a complex signal by adding two sinusoids and to simulate a realistic scenario by adding random noise.

Mathematical Background. Many real-world signals are composites of multiple simpler signals. By the principle of superposition, we can create a composite signal $y[n]$ by adding two signals, $x_1[n]$ and $x_2[n]$:

$$y[n] = x_1[n] + x_2[n] \quad (6)$$

Furthermore, measurement systems are never perfect and introduce random fluctuations, or noise, $\eta[n]$. A noisy signal $z[n]$ is modeled as:

$$z[n] = y[n] + \eta[n] \quad (7)$$

A common noise model is Additive White Gaussian Noise (AWGN), where each noise sample $\eta[n]$ is drawn from a normal (Gaussian) distribution with zero mean and a specified variance σ^2 .

Tasks.

1. Define parameters for two sinusoidal signals:
 - Signal 1: $f_1 = 10$ Hz, $A_1 = 1.0$
 - Signal 2: $f_2 = 25$ Hz, $A_2 = 0.5$
 - Use a sampling rate $f_s = 200$ Hz and a duration of 1 second.
2. Generate the two discrete-time signals, `x1_n` and `x2_n`.
3. Create the composite signal, `y_n`, by adding them together.
4. Generate Gaussian noise `noise` with the same length as your signal. Use `np.random.randn(num_samples)`. This function generates noise from a standard normal distribution (mean=0, variance=1). Scale the noise by a factor (e.g., 0.3) to control its power.
5. Create the final noisy signal, `z_n`, by adding the noise to `y_n`.
6. Create a single plot that shows:
 - The composite signal `y_n` (without noise) as a solid line.
 - The noisy signal `z_n` as a semi-transparent line on top of it.

This will help you visualize how the noise corrupts the original signal.

```

1 # Parameters
2 fs = 200
3 duration = 1.0
4 n = np.arange(duration * fs)
5
6 # Signal 1
7 f1, A1 = 10, 1.0
8 x1_n = A1 * np.sin(2 * np.pi * f1 * n / fs)
9
10 # Signal 2
11 f2, A2 = 25, 0.5
12 x2_n = A2 * np.sin(2 * np.pi * f2 * n / fs)

```

```
13
14 # Composite signal
15 y_n = x1_n + x2_n
16
17 # Generate noise
18 noise_amplitude = 0.3
19 noise = noise_amplitude * np.random.randn(len(n))
20
21 # Add noise to signal
22 z_n = y_n + noise
23
24 # Plotting
25 plt.figure(figsize=(12, 6))
26 plt.plot(n / fs, y_n, label='Clean Composite Signal')
27 plt.plot(n / fs, z_n, label='Noisy Signal', alpha=0.7)
28 plt.title('Composite Signal with Additive White Gaussian Noise')
29 plt.xlabel('Time (s)')
30 plt.ylabel('Amplitude')
31 plt.legend()
32 plt.grid(True)
33 plt.show()
```

Question. Adjust the `noise_amplitude` parameter to different values (e.g., 0.1, 1.0, 2.0). How does the level of noise affect your ability to visually discern the underlying structure of the composite signal?